Diagnosing the Client-Count Scaling Failure of Metric-Privacy Noise Calibration

Phase 1 Progress Report

Federated Learning + Differential Privacy Research Project

August 4, 2026

Abstract

This report documents Phase 1 ("Diagnosis") of an ongoing investigation into why the metric-privacy noise-calibration mechanism of Sáinz-Pardo Díaz et al. (2026), which showed a genuine accuracy advantage over fixed-multiplier global DP at 8 federated clients (+6.9 pp homogeneous, +12.2 pp non-IID), collapses or reverses at 48 clients. Two constant-compute experimental controls were designed, implemented, and run to disentangle a genuine scaling failure of the mechanism from a confound with per-round training budget. In the course of this work, a chain of five real infrastructure bugs was found and fixed (a unified-memory leak, a DataLoader worker-churn regression, a dataset-reload race, an uncaught aggregation crash, and a methodology confound in the first control design itself). Most significantly, a non-determinism in Apple Silicon's MPS backend was discovered and directly proven: re-running the *exact same configuration* (same seed, same hyperparameters, same code) at two different times produced accuracy deltas of up to 53.3 pp. This noise floor is comparable to or larger than the effect sizes this project has been treating as findings, which means **no single-run result produced on this machine to date — across either constant-compute design — can currently be trusted at face value**. Two of the highest-priority findings have already been directly noise-checked and both failed to hold up as stated. The report closes with the concrete next steps required before diagnosis can resume on solid ground.

Contents

1 Background and Research Question **1**

2 Summary of Work Completed **2**

3 Infrastructure Fixes **2**

4 Constant-Compute Methodology **3**
4.1 Design v1: round-scaled (superseded) 3
4.2 Design v2: rounds-fixed, epoch-scaled (corrected) 3

5 Results **4**
5.1 v1: round-scaled sweep 4
5.2 v2: rounds-fixed, epoch-scaled sweep 4

6 Critical Discovery: MPS Backend Non-Determinism **4**
6.1 How it surfaced . . . 4
6.2 Root cause . . . 6
6.3 Mitigation . . . 6

7 Noise-Floor Verification of Specific Findings **6**
7.1 Homogeneous / n=48 reversal (−18.91 pp reported) . . . 7
7.2 Non-IID / n=48 delta (+3.12 pp reported) . . . 7

8 Current Status **7**

9 Next Steps **8**

10 Conclusion **9**

1 Background and Research Question

The source paper calibrates server-side differential-privacy noise by the maximum pairwise distance between client model updates (`maximum_pairwise_model_distance` in `metricdp_pytorch/metricdp_strategy.py`) scaling the effective noise multiplier as noise_multiplier/distance. It reports this beating a fixed-multiplier "global-DP" baseline on accuracy at matched privacy, but validates the claim only at 4 clients, on one dataset/model, with an explicitly heuristic (unproven) calibration.

Prior work in this repository established two facts that motivate the current phase:

- At **8 clients**, sweeping the noise multiplier surfaces a genuine, previously unpublished effect: metric-privacy beats global-DP by +6.9 pp (homogeneous) and +12.2 pp (non-IID) at noise_multiplier = 0.05.
- At **48 clients**, reusing that same noise_multiplier = 0.05 under a *fixed* 20-round budget, the advantage shrinks to near zero or reverses (FedAvg +0.6/ + 2.2 pp, FedYogi −1.7/ − 0.2 pp), while overall accuracy drops sharply.

However, both the 8-client and 48-client sweeps hardcoded a 20-round training budget regardless of client count, while per-client data shrinks roughly $\propto 1/n$. This confounds two hypotheses: (a) the metric-privacy mechanism genuinely degrades as client count grows, or (b) each client is simply under-trained at 48 clients because a fixed round budget gives it far less data per round than at 8. **Phase 1's goal is to disentangle these two hypotheses** before any mechanism redesign (Phase 2) is attempted.

2 Summary of Work Completed

Work proceeded in three stages, each surfacing a problem that had to be resolved before the next could produce trustworthy data:

1. **Infrastructure hardening** (§3) — five real bugs found and fixed while running the first sweeps, none of which touch the research question directly but all of which were blocking reliable data collection.

2. **Constant-compute control design and execution** (§4, §5) — a first control design (v1) was implemented, run to completion (12 combinations), and written up; a methodology flaw in that design was then identified, leading to a corrected second design (v2), also run to completion (12 combinations).

3. **Measurement-noise investigation** (§6, §7) — comparing v1 and v2 exposed a much deeper problem than either was designed to detect: the training backend itself (Apple MPS) is non-deterministic across process launches even with every seed fixed, at a magnitude that can exceed the effect sizes being measured. This was root-caused, proven directly, and partially mitigated (an opt-in exact-reproducibility mode). Two specific findings were then re-run several times each to check whether they survive this noise; neither does, for different reasons.

The net result of this phase to date is not yet "does the mechanism fail at scale" — it is "the constant-compute controls needed to answer that question are built and run, but the platform they ran on is not yet proven precise enough to trust the answer they gave." §8 states this precisely.

3 Infrastructure Fixes

Running the first sweep at high client counts (48) and high round counts (240, see §4.1) immediately exposed problems invisible at the smaller scales used previously. All were root-caused and fixed on `feature/scaling-diagnosis` before any result in this report was collected.

Unified-memory leak (MPS).
Apple's unified-memory caching allocator does not return freed tensor memory to the OS. Flower's Ray simulation backend keeps each simulated client as a long-lived actor process for an entire run, so on MPS this pooled-but-idle memory accumulated round over round within each actor and eventually exceeded total system RAM on long, high-round-count runs. Fixed by calling `torch.mps.empty_cache()` / `torch.cuda.empty_cache()` once per client task.

Process orphaning.
Hard-killing hung runs during debugging left `torch_shm_manager` and other subprocess trees re-parented to `init` (`ppid=1`), which accumulated across debugging iterations and independently exhausted system memory/swap. Fixed with an explicit cleanup pass (matched via `ps -eo pid,ppid,args`, filtered on `ppid==1`) run before and after every sweep combination.

DataLoader worker churn.
`num_workers=2` was spawning and tearing down worker processes far more aggressively than expected under concurrent multi-actor load, driving CPU usage far above what the workload justified. Root cause was `num_workers` itself (not `persistent_workers`, which was briefly and incorrectly suspected and reverted after confirming it amortizes worker cost *across local epochs within one round*, not across rounds). Fixed by defaulting to `num_workers=0`.

Per-round dataset-reload race.
Evaluation artifacts were intermittently inconsistent with training-time metrics. Root-caused to the dataset being reloaded from disk for every client on every round under concurrent access; fixed with a process-level cache in `load_data_module()` keyed by `(factory_path, cache_dir)`.

Uncaught aggregation crash.
`ZeroDivisionError` in Flower's own clipping code could crash an entire run on a single collapsed (zero-norm) client update round. This is now caught, logged as a `metric-dp-aggregation-collapsed` flag, and the run continues — this flag is used directly in the diagnostics of §5.1.

4 Constant-Compute Methodology

4.1 Design v1: round-scaled (superseded)

The first control scaled the number of training rounds proportionally with client count, holding local epochs and batch size fixed at the paper's values:

$$\text{rounds}(n) = \text{round}\left(20 \cdot \frac{n}{4}\right), \qquad n \in \{4, 8, 48\}$$

so that each client's total local gradient-step count over its own data stays roughly constant across client counts (4 clients → 20 rounds; 48 clients → 240 rounds). The matrix was narrowed to {global-dp, metric-privacy} × {fedavg} × {homogeneous, non-IID} × {4, 8, 48} at noise_multiplier = 0.05 (the 8-client sweet spot) — 12 combinations, a deliberate time-budget reduction from the full paper-reproduction grid.

4.2 Design v2: rounds-fixed, epoch-scaled (corrected)

After analyzing v1's results (§5.1), a methodology flaw was identified: scaling *rounds* at fixed local epochs and batch size also scales *aggregation frequency* 12× from n=4 to n=48, while shrinking each client's per-round shard 12× — at 48 clients, clients see only 2–4 mini-batches per local epoch and are interrupted by aggregation and noise-injection 240 times instead of 20. Total gradient-step count is preserved, but the training dynamics are not comparable: the same number of steps delivered as many short, frequently-interrupted local phases is not equivalent to the same number of steps delivered as few, substantial local phases.

The corrected design instead holds rounds fixed at the paper's value and scales *local epochs*:

$$\text{rounds}(n) = 20 \text{ (constant)}, \qquad \text{local\_epochs}(n) = \text{round}\left(5 \cdot \frac{n}{4}\right), \qquad n \in \{4, 8, 48\}$$

This targets aggregation-frequency parity directly (every combination aggregates exactly 20 times) rather than only total-step-count parity. The same reduced 12-combination matrix was re-run under this design (`experiments/client_scaling/sweep_scale_controlled_epochs.py`, results in `results/scale_controlled_epochs/`).

5 Results

5.1 v1: round-scaled sweep

The headline v1 comparison against the earlier fixed-20-round n=48 baseline (`results/48client_scaling`):

At face value, v1 suggested the metric-privacy advantage *degrades further*, not less, once the round-budget confound is controlled for. As detailed in §6–§7, this headline result does not currently survive a direct noise check and should not be cited as established.

5.2 v2: rounds-fixed, epoch-scaled sweep

6 Critical Discovery: MPS Backend Non-Determinism

6.1 How it surfaced

v2's n=4 baseline uses *exactly* the same configuration as v1's n=4 baseline (both hold rounds fixed at 20 and local epochs fixed at 5 by construction, since $\text{round}(5 \cdot 4/4) = 5$) — same seed (42), same hyperparameters, same code path. These should be bit-for-bit reproducible runs. They are not.

Partition	Privacy	n	Rounds	Accuracy	Loss	F1	Collapsed rounds
homogeneous	global-dp	4	20	49.53%	1.0493	0.328	n/a
homogeneous	global-dp	8	40	14.06%	1.2760	0.035	n/a
homogeneous	global-dp	48	240	33.44%	0.4933	0.168	n/a
homogeneous	metric-privacy	4	20	70.94%	1.1314	0.691	0
homogeneous	metric-privacy	8	40	19.69%	54.858	0.065	14
homogeneous	metric-privacy	48	240	14.53%	0.5857	0.142	0
non-IID	global-dp	4	20	49.53%	1.0484	0.328	n/a
non-IID	global-dp	8	40	25.31%	1.3124	0.102	n/a
non-IID	global-dp	48	240	17.19%	1.1151	0.050	n/a
non-IID	metric-privacy	4	20	85.47%	0.5427	0.852	0
non-IID	metric-privacy	8	40	32.66%	23.472	0.161	23
non-IID	metric-privacy	48	240	20.31%	0.4509	0.069	0

Table 1: v1 (round-scaled) constant-compute sweep, all 12 combinations, noise_multiplier = 0.05, fedavg, seed 42. Source: `results/scale_controlled/`.

Partition	Δ (metric-privacy − global-dp), fixed 20 rounds	Δ, constant-compute (240 rounds)
homogeneous	+0.63 pp	**−18.91** pp
non-IID	+2.19 pp	+3.12 pp

Table 2: The metric-privacy advantage reverses hard for homogeneous data once the round budget is scaled to compensate for per-client data loss; it survives, barely, for non-IID.

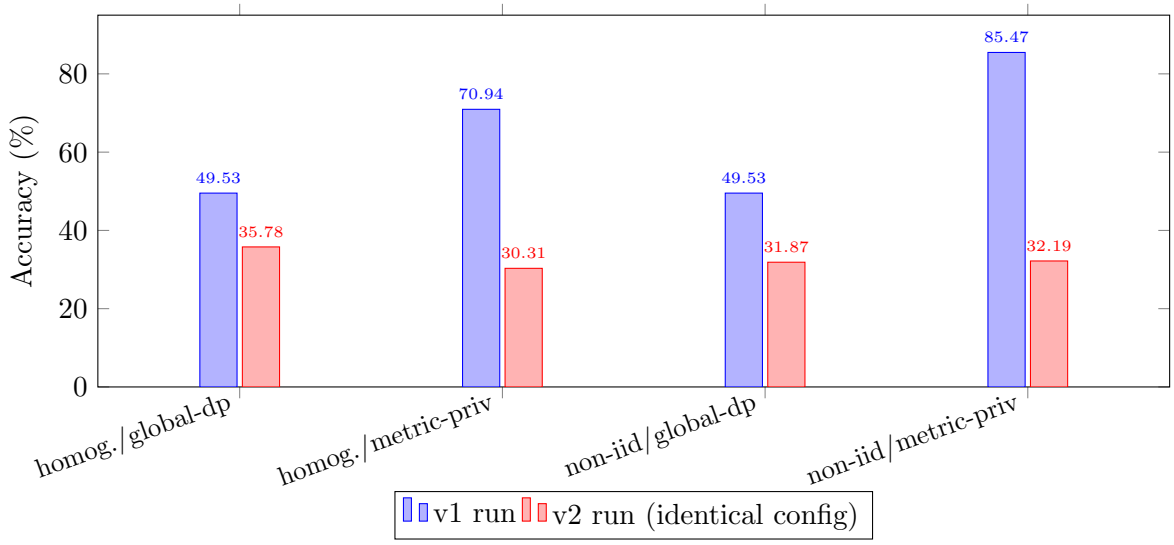

Figure 1: Same configuration, same seed, two different accuracies. The spread is largest exactly where v1's headline finding (metric-privacy's large n=4 advantage) lives.

This is, by a wide margin, the strongest and most direct evidence in this phase of work. It does not depend on interpreting small sample noise across a handful of extra reps (§7) — it is one configuration, one seed, compared against itself.

Partition	Privacy	n	Local epochs	Accuracy	Loss	Collapsed rounds
homogeneous	global-dp	4	5	35.78%	0.3699	n/a
homogeneous	global-dp	8	10	39.38%	0.5683	n/a
homogeneous	global-dp	48	60	35.47%	0.6191	n/a
homogeneous	metric-privacy	4	5	30.31%	0.5530	0
homogeneous	metric-privacy	8	10	24.22%	35.890	5
homogeneous	metric-privacy	48	60	34.53%	0.6054	0
non-IID	global-dp	4	5	31.87%	0.5600	n/a
non-IID	global-dp	8	10	38.44%	0.5443	n/a
non-IID	global-dp	48	60	33.44%	0.6816	n/a
non-IID	metric-privacy	4	5	32.19%	0.5797	0
non-IID	metric-privacy	8	10	24.84%	41.621	7
non-IID	metric-privacy	48	60	37.81%	0.5829	0

Table 3: v2 (epoch-scaled, corrected control) sweep, all 12 combinations, seed 42. Source: `results/scale_controlled_epochs/`. **Not yet written up as a standalone report** — this table is the first place these numbers appear.

Partition	Δ at n=4	Δ at n=8	Δ at n=48
homogeneous	−5.47 pp	−15.16 pp	−0.94 pp
non-IID	+0.32 pp	−13.60 pp	+4.37 pp

Table 4: v2's metric-privacy − global-dp deltas. Note this does *not* reproduce even the qualitative n=4 starting point of v1 (there: large positive advantage at n=4; here: near zero or negative) under an allegedly identical base configuration — see §6 for why.

6.2 Root cause

Confirmed directly (full writeup in `metricdp_pytorch/utils/device.py`'s `resolve_device` docstring): PyTorch's MPS backward pass is non-deterministic across separate process launches specifically when **multiple Ray client actors train concurrently on the shared MPS device**, even with every relevant seed fixed. This was isolated by elimination:

- Single-process, isolated runs reproduce exactly.
- Sequential (`max_parallel_clients=1`) runs reproduce exactly.
- Only concurrent multi-actor MPS training diverges.

The resulting weight divergence is real, not a display/formatting artifact: state_dicts differ by $\sim 5 \times 10^{-4}$ per parameter after a single round, versus CPU's $\sim 10^{-8}$ (float32's own precision floor, present on every backend and harmless). Two candidate fixes were tried and ruled out:

- `torch.use_deterministic_algorithms(True)` silently no-ops for the relevant MPS ops in this PyTorch version — zero warnings even with `warn_only=True`, meaning these ops are not hooked into the determinism-checking machinery at all.
- `PYTORCH_ENABLE_MPS_FALLBACK=1` caused the real training pipeline to hang (11+ minutes elapsed against ∼4s of actual CPU time — a genuine stall, not a slowdown).

Combination	v1 accuracy	v2 accuracy	Δ
homogeneous / global-dp	49.53%	35.78%	−13.75 pp
homogeneous / metric-privacy	70.94%	30.31%	−40.62 pp
non-IID / global-dp	49.53%	31.87%	−17.66 pp
non-IID / metric-privacy	85.47%	32.19%	−53.28 pp

Table 5: **Identical configuration, run at two different times, same seed**: all four n=4 baseline points diverge by 13.75–53.28pp. Metadata for both runs was diffed directly (num_clients, rounds, local_epochs, noise_multiplier, seed, partition, privacy, aggregation, batch_size all confirmed identical) — this is not a configuration bug, it is the training backend itself failing to reproduce.

6.3 Mitigation

CPU reproduces exactly (confirmed by metrics matching to full displayed precision across repeated runs) but is measured directly to be $\sim 3\times$ slower than MPS even at matched Ray parallelism (an n=4/20-round run: >11 minutes on CPU vs. MPS's 214.7–220.0s). Given most work in this repository is exploratory, CPU was made an *opt-in* override (`METRICDP_FORCE_CPU=1`) rather than the default — MPS stays default for speed, and CPU is reserved for runs whose numbers need to go into a committed report or cross-run comparison, where correctness matters more than wall-clock time. This tradeoff, and the full investigation above, is documented directly in `resolve_device()`'s docstring for future reference.

7 Noise-Floor Verification of Specific Findings

Given §6's proof that the noise floor is real and can reach tens of percentage points, every single-seed accuracy delta produced on this machine's MPS backend to date is suspect until checked. Rather than re-run the entire matrix under `METRICDP_FORCE_CPU=1` immediately (a large time cost), a cheaper interim methodology was used first: run 2 extra MPS repetitions of each side of a specific finding, and check whether the reported delta is bigger than the spread across those repetitions. This gives a fast, informal directional read without settling on a trustworthy magnitude. Two of v1's findings have been checked this way so far.

7.1 Homogeneous / n=48 reversal (−18.91 pp reported)

Privacy mode	v1 (original)	rep 1	rep 2
global-dp	33.44%	63.12%	57.34%
metric-privacy	14.53%	28.91%	39.38%

Table 6: Homogeneous n=48 noise-floor check, 2 extra MPS reps per arm. Source: `results/noise_floor_check/`.

Both arms swing enormously on their own — global-dp ranges 33.44–63.12%, metric-privacy 14.53–39.38%, across just 3 runs each. Paired within the same run generation, the delta stayed negative all 3 times (−18.91, −34.22, −17.97 pp) — some directional consistency — but crossing generations (comparing one generation's global-dp against a *different* generation's metric-privacy, which is exactly the risk a single-seed comparison runs) it ranges from −48.59 pp to +5.94 pp, including a sign flip.

Verdict: weak directional support only. The −18.91 pp magnitude is not trustworthy and should not be cited. Large effect, even larger noise.

7.2 Non-IID / n=48 delta (+3.12 pp reported)

Privacy mode	v1 (original)	rep 1	rep 2
global-dp	17.19%	16.41%	17.97%
metric-privacy	20.31%	21.41%	14.69%

Table 7: Non-IID n=48 noise-floor check, 2 extra MPS reps per arm. Source: `results/noise_floor_check_noniid/`.

Individual accuracies are far more stable here (global-dp 16.41–17.97%, metric-privacy 14.69–21.41% — a few points of spread, not tens) but the reported delta is small enough to be within that smaller noise anyway. Paired deltas: +3.12, +5.00, **−3.28** pp — the sign flips on the third pair.

Verdict: not established, via a different mechanism than the homogeneous case — there, a large effect was swamped by even larger noise; here, a small effect is swamped by smaller but still comparable noise.

8 Current Status

- All planned infrastructure fixes are in place and covered by the test suite (**47 passed**, 5 reproducibility-marker tests deselected by default, current as of this report).
- Both constant-compute control designs (v1 and v2) are fully implemented and have each completed their full 12-combination matrix. v2's results have not yet been written up as their own report (Table 3 is the first place they appear in narrative form).
- The MPS non-determinism is root-caused, directly proven (Table 5), and partially mitigated via an opt-in exact-reproducibility mode.
- Two specific findings have been noise-checked against this now-confirmed noise floor. **Both failed to hold up as originally stated**, for two different mechanisms (large effect swamped by larger noise; small effect swamped by comparable noise).
- **Every other single-seed result produced so far is unverified**, including: v1's n=8 points, v1's n=4 baseline (now known to be unstable per Table 5), all of v2 (Table 3), and the original `8client_scaling`/`48client_scaling` /`noise_sweep` result sets that motivated this phase in the first place. Given how differently the two checked findings behaved (huge individual-run noise vs. small individual-run noise), the noise magnitude found for one finding does *not* generalize to another — each remaining single-seed number needs its own check before being cited.

Bottom line for this reporting period: Phase 1's stated goal (disentangle round-budget confound from genuine scaling failure) is *methodologically ready to answer* — both a flawed and a corrected constant-compute design exist and have both been run — but *not yet answerable with confidence*, because the platform used to generate every data point so far has a proven noise floor of the same order of magnitude as the effects being measured. The diagnosis has not failed; it has correctly stalled at the point where continuing without addressing measurement precision would risk drawing conclusions from noise.

9 Next Steps

Three options were identified for how to proceed, and a decision is needed before further sweep work continues:

1. **Continue noise-checking existing findings** (2 extra MPS reps per remaining finding, ∼3.5–4.5h each) — cheap per check, purely retrospective, tells us which of the already-collected numbers to trust or discard, but produces no new diagnostic progress.
2. **Proceed to the remaining Phase 1 roadmap items** (per-client-count noise-multiplier resweep; instrumenting the raw pairwise-distance distribution per round; failure-mode/NaN logging in the runner) on the current MPS pipeline, accepting that these new numbers will carry the same unresolved noise until separately checked.
3. **Switch to `METRICDP_FORCE_CPU=1` for all new reproducibility-critical runs going forward**, trading ∼3× wall-clock time for exact reproducibility on every future data point, instead of reactively spot-checking after the fact.

Recommendation to be finalized with the supervisor: given how central measurement precision has turned out to be to this phase, item 3 — forcing CPU for anything that will be cited as a result — is the most defensible path for the remainder of Phase 1, reserving MPS for fast exploratory iteration only.

10 Conclusion

This phase set out to answer a scoping question (is the metric-privacy scaling failure real, or an artifact of a fixed round budget) and, in the process of building the infrastructure to answer it rigorously, uncovered a more fundamental problem: the experimental platform itself was not precise enough to trust the answer. That discovery is itself a legitimate and useful Phase 1 outcome — it prevents a false conclusion from being carried into Phase 2's mechanism redesign, where it would be far more expensive to unwind. The corrected constant-compute control (v2), the confirmed and documented MPS non-determinism, and the two completed noise-floor checks together define exactly what further work (per §8) is required before Phase 1's central question can be answered with confidence.